

UNIVERSIDADE PRESBITERIANA MACKENZIE

Projeto Aplicado III – Sistemas de Recomendação

APÊNDICE TÉCNICO

Código-Fonte e Instruções de Reprodução

GRUPO 19

Aline Souza Aprelini – RA: 10369307

São Paulo – 2026

ESTRUTURA DO PROJETO

Este apêndice documenta a estrutura técnica do projeto e fornece o código-fonte completo para reprodução dos resultados.

Arquivos do Projeto

Estrutura de Arquivos

```
projeto-recomendacao-cursos/  
├── RECOMENDACAO_CURSOS.ipynb      # Notebook principal (Jupyter)  
├── README.md                      # Documentação básica  
├── requirements.txt                # Dependências Python  
└── dados/  
    └── (carregado via URL GitHub)
```

Requisitos do Sistema

Ambiente de Desenvolvimento

- Python 3.8 ou superior
- Jupyter Notebook ou Google Colab
- Conexão com internet (para carregar dados)

Bibliotecas Necessárias

Arquivo requirements.txt:

```
pandas==2.0.0  
numpy==1.24.0  
scikit-learn==1.3.0  
matplotlib==3.7.0  
seaborn==0.12.0
```

Instalação via pip:

```
pip install -r requirements.txt
```

Fonte dos Dados

Base de dados pública de cursos da plataforma Coursera, disponibilizada no GitHub por RY05 (2026).

URL: <https://raw.githubusercontent.com/ry05/couReco/master/data/coursera-courses.csv>

- Formato: CSV
- Tamanho: ~1.000 registros de cursos
- Variáveis principais: course_name, skills, description, course_rating, enrolled_student_count, course_difficulty

CÓDIGO-FONTE COMPLETO

Notebook: RECOMENDACAO_CURSOS.ipynb

Célula 1 – Importação de Bibliotecas

Importação das bibliotecas necessárias para análise de dados, machine learning e visualização.

```
import pandas as pd
import numpy as np
from sklearn.neighbors import NearestNeighbors
from sklearn.metrics import mean_squared_error
from sklearn.feature_extraction.text import TfidfVectorizer
import matplotlib.pyplot as plt
import seaborn as sns
```

Célula 2 – Carregamento dos Dados

Carrega a base de dados de cursos diretamente do repositório GitHub.

```
df =
pd.read_csv('https://raw.githubusercontent.com/ry05/couReco/master/data/c
oursera-courses.csv')
df.head(10)
```

Célula 3 – Análise de Valores Ausentes

Verifica a presença de valores nulos em cada coluna da base de dados.

```
df.isna().sum()
```

Célula 4 – Visualização de Valores Nulos

Gera um mapa de calor (heatmap) para visualizar a distribuição de valores ausentes na base.

```
sns.heatmap(df.isna(), cbar=False)
plt.title("Mapa de valores nulos")
plt.show()
```

Célula 5 – Remoção de Colunas com Muitos Nulos

Remove as colunas com alta taxa de valores ausentes (>75% nulos).

```
df.drop([
    'percentage_of_new_career_starts',
    'percentage_of_pay_increase_or_promotion'
], axis=1, inplace=True)
```

Célula 6 – Imputação de Valores Nulos

Preenche valores ausentes nas colunas numéricas. Para `course_rating` usa-se a média; para contagens usa-se zero.

```
df['course_rating'] =
df['course_rating'].fillna(df['course_rating'].mean())
df['course_rated_by'] = df['course_rated_by'].fillna(0)
df['enrolled_student_count'] = df['enrolled_student_count'].fillna(0)
```

Célula 7 – Verificação Pós-Tratamento

Confirma que não há mais valores nulos após o tratamento.

```
df.isna().sum()
```

Célula 8 – Vetorização TF-IDF

Aplica a técnica TF-IDF sobre a coluna 'skills' para transformar os textos em vetores numéricos.

```
tfidf = TfidfVectorizer(stop_words='english')
```

```
matriz = tfidf.fit_transform(df['skills'])
```

Célula 9 – Treinamento do Modelo KNN

Treina o modelo K-Nearest Neighbors com métrica de similaridade do cosseno.

```
modelo = NearestNeighbors(metric='cosine', algorithm='brute')
modelo.fit(matriz)
```

Célula 10 – Função de Recomendação

Implementa a função que recebe um nome de curso e retorna os N cursos mais similares.

```
def recomendar_cursos(nome_curso, n=5):
    if nome_curso not in df['course_name'].values:
        return "Curso não encontrado"

    idx = df[df['course_name'] == nome_curso].index[0]
    distancias, indices = modelo.kneighbors(matriz[idx],
n_neighbors=n+1)
    recomendados = df.iloc[indices[0][1:]]['course_name']

    return recomendados

# Teste
recomendar_cursos("Data Science")
```

INSTRUÇÕES DE REPRODUÇÃO

Este guia passo a passo permite que qualquer pessoa reproduza os resultados do projeto.

Opção 1: Google Colab (Recomendado)

1. Acesse <https://colab.research.google.com>
2. Crie um novo notebook ou faça upload do arquivo RECOMENDACAO_CURSOS.ipynb
3. Execute as células sequencialmente (Runtime > Run all)
4. Os dados serão carregados automaticamente da URL do GitHub

☒ Vantagens do Google Colab: sem configuração local, ambiente pronto, gratuito!

Opção 2: Ambiente Local

5. Clone ou baixe os arquivos do projeto
6. Instale as dependências: `pip install -r requirements.txt`
7. Inicie o Jupyter Notebook: `jupyter notebook`
8. Abra o arquivo RECOMENDACAO_CURSOS.ipynb
9. Execute as células sequencialmente

Opção 3: Executar como Script Python

Para uso sem Jupyter, o código pode ser adaptado para um script .py:

```
# recomendacao.py
import pandas as pd
from sklearn.neighbors import NearestNeighbors
from sklearn.feature_extraction.text import TfidfVectorizer

# [código completo aqui]

if __name__ == "__main__":
    resultado = recomendar_cursos("Machine Learning", n=5)
    print(resultado)
```

Execução via terminal:

```
python recomendacao.py
```

Testando o Sistema

Após executar o notebook, teste a função de recomendação com diferentes cursos:

```
# Exemplos de teste
recomendar_cursos("Machine Learning")
recomendar_cursos("Python for Everybody")
recomendar_cursos("Data Science")
recomendar_cursos("Business Analytics")
```

NOTAS TÉCNICAS ADICIONAIS

Decisões de Implementação

Escolha do TF-IDF Básico

A implementação utiliza `TfidfVectorizer` com configuração padrão, exceto pela remoção de stopwords em inglês. Essa configuração foi escolhida por:

- Simplicidade e clareza didática
- Boa performance para o tamanho da base de dados
- Foco na prova de conceito

Algoritmo KNN com 'brute'

O parâmetro `algorithm='brute'` força a busca exaustiva de vizinhos. Embora menos eficiente que estruturas de dados como KD-tree ou Ball-tree, foi escolhido porque:

- Garante resultados exatos (não aproximados)
- Base de dados pequena (~1.000 registros) torna o impacto desprezível
- Métrica 'cosine' não é otimizada em estruturas de árvore

Número Padrão de Recomendações

A função `recomendar_cursos` usa `n=5` como padrão, mas aceita qualquer valor. Na documentação principal foram testados valores `k=5`, `k=10` e `k=15` para análise de sensibilidade.

Limitações da Implementação Atual

- Não calcula métricas de avaliação (`Precision@K`, `Recall@K`) automaticamente
- Não possui interface gráfica ou API para uso externo
- Depende de conexão à internet para carregar dados
- Não salva o modelo treinado (retreina a cada execução)

Melhorias Propostas no Código

Para trabalhos futuros, as seguintes melhorias técnicas são sugeridas:

10. Implementar cálculo automático de `Precision@K` e `Recall@K`
11. Adicionar parâmetros opcionais ao `TfidfVectorizer` (`ngram_range`, `min_df`, `max_df`)
12. Salvar e carregar modelo treinado via `pickle/joblib`

13. Criar API REST com Flask ou FastAPI
14. Adicionar logging e tratamento de erros robusto
15. Implementar testes unitários